Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 6

Critical Section and Synchronization

Race conditions, the critical-section problem, Peterson's solution, hardware support and mutex locks.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Describe the critical-section problem and show what a race condition is.
2. State the three requirements a solution must meet.
3. Explain Peterson's solution and why it is correct.
4. Describe hardware solutions: test-and-set, compare-and-swap and atomic variables.
5. Use a mutex lock to protect a critical section.

THIS WEEK

Topic 6

Processes and threads share data, and the scheduler can interrupt them at any instruction. This week is about staying correct anyway.

DISCUSSION

Two people withdraw from one account at the same moment

The balance is 100. Each withdraws 80 at a different machine. Both machines read 100 first. What is the balance afterwards?

PART 1

The problem

Concurrent access to shared data may leave the data inconsistent.

Background

Processes run concurrently

A process may be interrupted at any time, with its work only partly done.

They share data

Threads of one process share its memory. Cooperating processes share memory regions, files and kernel structures.

The result can depend on timing

If two of them change the same data, the outcome may depend on the exact order in which their instructions run.

A race condition

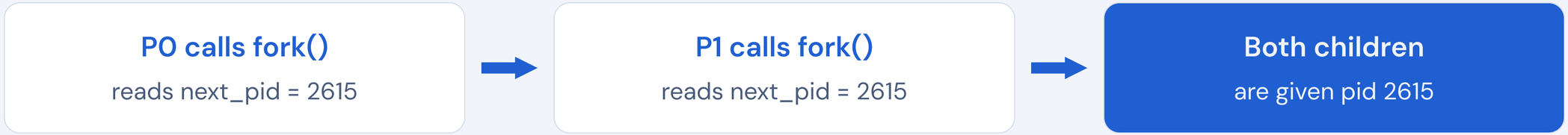
Step	Machine A	Machine B	Balance
1	reads 100		100
2		reads 100	100
3	computes 100 - 80		100
4		computes 100 - 80	100
5	writes 20		20
6		writes 20	20

160 was paid out. The balance fell by only 80.

Each machine did its own work correctly. The error is in the interleaving.

A race condition: several processes access and change the same data concurrently, and the result depends on the order of access.

The kernel has the same problem



The kernel keeps the next available process identifier in one variable. Unless access to it is controlled, two processes can receive the same identifier.

The critical-section problem

```
while (true) {

    /* entry section */

    /* critical section */

    /* exit section */

    /* remainder */
}
```

Critical section

The segment of code where a process changes shared data: variables, a table, a file.

The rule

When one process is in its critical section, no other process may be in its own.

The problem

Design a protocol for this. Each process asks permission in an entry section and gives it up in an exit section.

Three requirements for a solution

01

Mutual exclusion

If a process is executing in its critical section, no other process can be executing in its critical section.

02

Progress

If no process is in its critical section and some want to enter, the choice of who enters next cannot be postponed forever. Only those not in their remainder section take part in the choice.

03

Bounded waiting

After a process asks to enter, there is a limit on how many times others may enter before it does.

We assume each process runs at some speed above zero, and assume nothing about their relative speeds.

A first idea: disable interrupts

SINGLE CORE

The idea

The entry section disables interrupts and the exit section enables them. Nothing can preempt the process in between.

THE LIMITS

Why it is not enough

A critical section that runs for a long time stops everything else, and some process may never get in. On a multiprocessor it does not work at all: another core can still run the other process.

PART 2

Software solutions

Two processes, shared variables, and no special hardware.

Attempt 1: take turns

```
int turn = 0; /* shared */

/* process i; the other is j */
while (true) {
    while (turn == j)
        ; /* wait */

    /* critical section */

    turn = j;
    /* remainder */
}
```

The variable turn says whose turn it is to enter.

Mutual exclusion holds: turn has one value, so only one process gets past the loop.

Progress does not hold. The processes must alternate strictly. If process j never wants to enter again, process i is locked out after its next turn.

Peterson's solution

```
int turn;
bool flag[2] = {false, false};

/* process i; the other is j */
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ; /* wait */

    /* critical section */

    flag[i] = false;
    /* remainder */
}
```

flag[i]

Process i says: I am ready to enter.

turn

Then it politely gives the turn to the other. If both do this at once, the later assignment to turn wins, and decides who waits.

Why Peterson's solution is correct

Mutual exclusion

Process i enters only if flag[j] is false or turn is i. Both flags are true when both want in, and turn cannot be 0 and 1 at once.

Progress

Process i waits only while j is ready and it is j's turn. If j is not interested, flag[j] is false and i enters at once.

Bounded waiting

When j leaves, it sets flag[j] to false. If it comes back it sets turn to i. So i enters after at most one entry by j.

Peterson's solution on modern hardware



Not guaranteed to work

To run faster, processors and compilers may reorder operations that do not depend on each other. With one thread the result is the same. With two it may not be.



What can go wrong

If the two assignments in the entry section are reordered, both processes can be in their critical sections at the same time.



Why we still study it

It is a clear algorithm, and proving it correct shows exactly what mutual exclusion, progress and bounded waiting mean.

PART 3

Hardware support

Instructions that the processor guarantees to execute as one uninterruptible unit.

Three forms of hardware support

01

Memory barriers

An instruction that forces every change made in memory so far to become visible to all other processors before execution continues.

02

Hardware instructions

Test a word and set it, or compare two words and swap them, atomically: as one step that cannot be interrupted.

03

Atomic variables

Variables whose basic updates, such as an increment, are atomic. Built on the hardware instructions.

A memory model describes what a processor guarantees. Strongly ordered: a change is visible to other processors at once. Weakly ordered: it may not be.

test_and_set

```
/* executed atomically */
bool test_and_set(bool *target) {
    bool old = *target;
    *target = true;
    return old;
}

bool lock = false; /* shared */

while (test_and_set(&lock))
    ; /* wait */
/* critical section */
lock = false;
```

It returns the old value of the word and sets the word to true, in one step.

If the lock was free, the caller gets false back and has taken the lock. If it was taken, the caller gets true back and tries again.

Two processes executing it at the same moment are run one after the other, in some order.

compare_and_swap

```
/* executed atomically */
int compare_and_swap(int *value,
                     int expected, int new_value) {
    int old = *value;
    if (*value == expected)
        *value = new_value;
    return old;
}

int lock = 0; /* shared */

while (compare_and_swap(&lock, 0, 1) != 0)
    ; /* wait */
/* critical section */
lock = 0;
```

It changes the word only if the word still holds the value we expect.

It always returns the original value, so the caller can tell whether the swap happened.

The first process to call it finds 0, sets 1 and enters. The others find 1 and wait.

Adding bounded waiting

THE GAP

What is missing

The two locks above give mutual exclusion and progress. They do not give bounded waiting: a process may lose the race for the lock again and again.

TAKE TURNS

The fix

Keep an array waiting[n], one entry per process. On leaving, a process does not simply free the lock. It scans the array in circular order and hands the critical section to the next process that is waiting.

Any waiting process then enters within n - 1 turns.

Atomic variables

```
void increment(atomic_int *v) {
    int temp;
    do {
        temp = *v;
    } while (temp !=
             compare_and_swap(v, temp, temp + 1));
}
```

An update that cannot be interrupted, with no lock.

Read the value. Try to replace it by the value plus one. If another thread changed it in between, the swap fails and we try again.

This removes the race on a single counter. It does not remove races that involve several variables or a condition such as the buffer being empty.

PART 4

Mutex locks

The hardware instructions are for kernel designers. Application programmers need something simpler.

The mutex lock

```
acquire() {
    while (!available)
        ; /* busy wait */
    available = false;
}

release() {
    available = true;
}

acquire(lock);
/* critical section */
release(lock);
```

Mutex is short for mutual exclusion.

A boolean variable says whether the lock is available. A process acquires the lock before its critical section and releases it after.

acquire() and release() must themselves be atomic. They are built with compare-and-swap.

Spinlocks



Busy waiting

A process that cannot get the lock loops, calling acquire() again and again. This kind of lock is called a spinlock. The loop uses CPU time that another process could have used.



When a spinlock is good

No context switch is needed. On a multicore system, if the lock is held for a short time, spinning on one core while the holder finishes on another is cheaper than two context switches.

A rule of thumb: use a spinlock if the lock is held for less than the time of two context switches.

The account, protected

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void withdraw(int amount) {
    pthread_mutex_lock(&m);
    if (balance >= amount)
        balance = balance - amount;
    pthread_mutex_unlock(&m);
}
```

Back to the opening question.

Reading the balance, testing it and writing it back are now one critical section.

Machine B cannot read the balance until machine A has finished. It then finds 20, and the second withdrawal of 80 is refused.

Key takeaways

01

A race condition

The result depends on the order in which concurrent processes touch shared data.

02

Three requirements

Mutual exclusion, progress and bounded waiting.

03

Atomic hardware instructions

test-and-set and compare-and-swap are the foundation of every lock.

04

Mutex locks

acquire before the critical section, release after. A spinlock waits by looping.

Next session

COMING UP

After the midterm: Synchronization I

Week 7 is the review and Week 8 the midterm. Week 9 continues with semaphores, monitors and liveness.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 6, sections 6.1 to 6.5.